Homework 31

The Rabbit Hunt

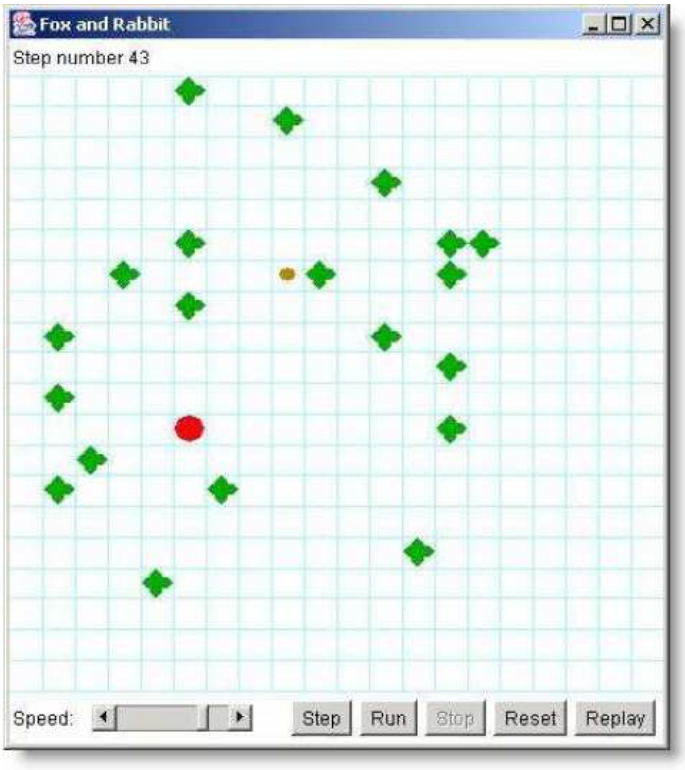
This homework assignment requires you to work with multiple classes as well as loops and decisions. The scoring is a little different than a regular homework assignment, though. You can earn up to 5 points of extra credit. When you complete the program, your grade will be based on the percent of times that the rabbit escapes from the fox. That means you'll get 10 points on the assignment if your rabbit escapes 100% of the time and 5 points if he escapes 50% of the time. In past semesters, the best students have managed about a 95% survival rate. There is one additional caveat: you can only modify the code in the Rabbit class; you can't shoot, drug or otherwise make the Fox stupid.

Here's the story:

A **fox** is hunting a **rabbit** in a field. The field contains a number of **bushes** which obstruct both the fox's view and the rabbit's view, so each may or may not be able to see the other. The fox tries to catch the rabbit; the rabbit tries to get away from the fox. If the fox can catch the rabbit, he eats it (and wins). If the rabbit can keep away from the fox for 100 turns, the rabbit wins.

You are the rabbit.

Getting started:



Download the **RabbitHunt.jar** file from the exercise page and then import it as a project into Eclipse. When you run the program, you'll see something like this:

The big red dot represents the fox, the small brown dot is the rabbit, and the green blobs are bushes.

Run the **RabbitHunt** program a few times and see what happens. Notice that you can step through the hunt, or just let it run. You can adjust the speed at which the animation proceeds (this does not affect what happens, only how quickly it happens). Notice that the rabbit almost always meets an untimely end. However, occasionally the rabbit will evade the fox for 100 turns, and the rabbit wins--this doesn't happen very often.

Now look at the `Rabbit` class. It has a constructor `Rabbit(model, row, column)`,

and it has a method **decideMove()**, which decides how the rabbit is going to move at each turn. The rabbit I've given you is a really stupid rabbit--it just moves randomly.

Your job is to improve the **Rabbit** class so that the rabbit escapes more often. This is your class; you can do (almost) anything you want with it. The **Rabbit** class is the only one you are allowed to change. It is probably impossible to find a strategy for the rabbit that ensures a 100% survival rate.

Additional Information about the Structure of the program

This program uses the Model-View-Controller design pattern. That is, there are separate classes that do the following: The **Model** represents the "rules of the game." The **View** displays what is going on. The **Controller** is the part of the program that displays the controls (the four buttons and the speed controls at the bottom of the window).

The **Controller** and **View** classes are almost entirely independent of the **Model** class. The **Rabbit** class is part of the Model. This means that you do not have to understand anything about either the Controller or the View in order to complete your assignment.

In addition to the **Model** class, there are **Animal**, **Rabbit**, **Fox**, and **Bush** classes. **Rabbit** and **Fox** are subclasses of **Animal**; all other classes just extend **Object**. This is the part of the program that you will work with. I'll describe these classes briefly, just to get you started; but you should examine the code to find out how it all really works and fits together.

The **Model** class places the fox, rabbit, and bushes in the field and then:

- gives the rabbit and the fox each a chance to move (one moves, then the other--they don't both move at the same time)
- tells the **View** to display the result of these two moves, and determines which animal won

In addition, **Model** provides several static constants that you can use:

`Model.N`	indicates "north" (straight up)
`Model.NE`	indicates "northeast" (up and to the right)
`Model.E`	indicates "east" (to the right)
`Model.SE`	indicates "southeast" (to the right and down)
`Model.S`	indicates "south" (straight down)
`Model.SW`	indicates "southwest" (to the left and down)
`Model.W`	indicates "west" (to the left)
`Model.NW`	indicates "northwest" (up and to the left)
`Model.STAY`	indicates "no move"
`Model.MIN_DIRECTION`	the numerically smallest direction (not including `STAY`)
`Model.MAX_DIRECTION`	the numerically largest direction (not including `STAY`)

`Model.BUSH`	indicates a bush
`Model.FOX`	indicates a fox
`Model.RABBIT`	indicates a rabbit
`Model.EDGE`	indicates the edge of the board

Finally, **`Model`** provides the following method:

`static int turn(int direction, int amount)`

> Given a starting direction and the number of 1/8 turns to make *clockwise*, this method returns the resultant direction.

Other classes that you'll work with are:

- **`Bush`**: which doesn't doesn't do anything; the bushes just sit there and get in the way.
- **`Animal`**: provides methods that are the same for both the rabbit and the fox: looking in a particular direction, measuring the distance to an object, and moving.
- **`Fox`**: tries to catch and eat the rabbit.
- **`Rabbit`**: is an animal that tries not to get caught and eaten.

The **`Animal`** class provides the following methods that are inherited by **`Fox`** and **`Rabbit`**.

`int look(int direction)`
> Return one of the constants **`Model.BUSH`**, **`Model.FOX`**, **`Model.RABBIT`**, or **`Model.EDGE`**, depending on what the animal sees in that direction.

`int distance(int direction)`
> Returns the number of steps the animal would have to make in order to land on top of the nearest object (or go off the edge of the playing area).

`boolean canMove(int direction)`
> Tells whether the animal can move in the given direction without being stopped by a bush or the edge of the board. Doesn't say whether it's a good idea to move that direction.

The part you have to write:

When it is the rabbit's turn to move, the model sends the message **`decideMove()`** to the rabbit. The rabbit must return a direction in which to move. I have written a **`Rabbit`** class, but my rabbit makes really stupid moves. Your job is to fix the **`Rabbit`** class by replacing my **`decideMove()`** method with a better one.

Note that the rabbit does not actually move itself. Instead, it responds to the **`decideMove()`** message by returning the direction in which it wants to move. The model will then move the rabbit in that direction, if the move is possible and legal. If the move cannot be made (because of a bush or the edge of the board), the rabbit will not move.

Run the Grader class to quickly execute the program a few hundred times and to get an overall percentage. When you're done, follow the instructions on the assignment Web page to submit your program.